Memory Slice (.msl)

Binary Format Specification

Version: 1.0.0
Status: Working Draft
Editor(s): Daniel Baier
This version: https://github.com/MemorySlice/memslice-spec/releases/tag/v1.0.0
Latest version: https://github.com/MemorySlice/memslice-spec/

Abstract

Memory Slice (`.msl`) is a self-describing, block-based binary format for capturing the forensic state of a single operating system process. It records both the virtual address space (with per-page acquisition status) and transient OS-queryable metadata that exists only while the process is alive. This document specifies the binary layout: file header, block architecture, integrity chain, capture-time payloads, cross-referencing, capability bitmap, raw dump import mechanism, investigation mode with system-wide context capture, and full-container AEAD encryption with post-quantum key encapsulation support.

This specification accompanies the research paper *Memory Slice: A Process-Centric Dump Format for Differential Memory Analysis and Volatility 3 Plugin Synthesis.*

Contents

1 Scope and Conventions

1.1 Scope

This document defines the binary wire format of Memory Slice (`.msl`) files. It does *not* specify acquisition procedures or analysis algorithms.

1.2 Normative Language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, "RECOMMENDED", **MAY**, and **OPTIONAL** are per RFC 2119 [1].

1.3 Encoding Conventions

- All multi-byte structural integers are **little-endian**. The file header's `Endianness` byte at offset `0x08` **MUST** be `0x01` in version 1.1. The field is retained for forward compatibility; future versions **MAY** define `0x02` (big-endian). Consumers encountering a value other than `0x01` **MUST** reject the file.
- Strings are UTF-8, null-terminated, padded to **8-byte** boundaries with zero bytes. The pad function is defined as $\text{pad8}(n) = \lceil n/8 \rceil \times 8$, where n is the byte length of the UTF-8 encoded string **including** the mandatory terminating NUL (`0x00`). All padding bytes **MUST** be `0x00`. Textual string fields **MUST NOT** contain `0x00` bytes before the terminating NUL.
- Where a string field is accompanied by an explicit length field (e.g., `PathLen`, `ExePathLen`), the **length field is authoritative**: it specifies the byte count including the terminating NUL. The on-disk storage occupies pad8(Len) bytes. Consumers **MUST** use the length field to determine string extent and **MUST NOT** scan beyond `Len` bytes. The NUL byte at position Len − 1 serves as a consistency check.
- Producers **MUST** emit well-formed UTF-8 per RFC 3629 [3]. Consumers encountering invalid UTF-8 sequences **SHOULD** preserve the raw bytes and **MAY** flag the field as malformed, but **MUST NOT** reject the entire block. This ensures that forensic evidence is never silently discarded due to encoding errors in captured metadata.
- Variable-length fields (`PageStateMap`, `NativeBlob`) are padded to 8-byte boundaries.
- UUIDs **MUST** be version 4 (random) per RFC 9562 [2], stored as 16 raw bytes in RFC 9562 binary layout. See Section 1.5.
- Integrity hashes use the algorithm selected by `HashAlgo` (Section 4.4); all registered algorithms produce 32-byte output. The default is BLAKE3 [4].
- Timestamps: unsigned 64-bit nanoseconds since Unix epoch (1970-01-01T00:00:00Z).
- All `Reserved` fields **MUST** be zero (producers) and **MUST** be ignored (consumers).

1.4 Byte Order Domains

Three byte-order regimes coexist within an MSL file. Implementers **MUST** observe the boundaries precisely:

1. **Structural integers.** All multi-byte integers in the file header, block headers, and payload fixed fields are little-endian (controlled by `Endianness`=`0x01`).

2. **UUIDs.** All UUID fields are stored in RFC 9562 binary layout (big-endian mixed-endian per the RFC) and are *never* subject to byte-swapping, regardless of the file's `Endianness` setting.
3. **Captured memory content.** Bytes within `PageData` are stored verbatim in the target architecture's native byte order; the format does not reorder them.

1.5 UUID Requirements

All UUID fields (`DumpUUID`, `BlockUUID`, `ParentUUID`, payload UUIDs) **MUST** be version 4 (random) per RFC 9562. The 4-bit version field (bits 48–51) **MUST** be `0100`; the 2-bit variant (bits 64–65) **MUST** be `10`. UUIDs are stored as 16-byte sequences in RFC 9562 binary layout and are never subject to byte-swapping.

Implementation guidance: UUID generation

A thread-local fast PRNG such as `xoshiro256++` or `PCG64`, seeded once from OS entropy, provides sufficient uniqueness ($\sim 2^{-61}$ collision probability per pair) while avoiding CSRNG overhead per block. Note that this is a *performance optimization*, not a conformance requirement; implementations **MAY** use a CSRNG if preferred.

1.6 Terminology

Producer
Software that creates or appends blocks.

Consumer
Software that reads an MSL file.

Acquirer
A producer capturing live process state (types `0x0000`–`0x0FFF`).

Importer
A producer converting a raw dump into MSL (Section 11).

Block
Fixed header + variable payload.

Payload
Block-type-specific data after the header.

Preamble
Block-level fields at the start of a payload that precede repeated entry structures (e.g., `EntryCount`).

Investigation Mode
Acquisition for forensic investigations with system-wide context (Section 6).

Analysis Mode
Default mode for research and tool development. No system-wide context required.

The following sections define the format itself, starting with the high-level file structure.

2 Format Overview

An MSL file is a linear byte stream: a **file header** followed by **typed, length-prefixed blocks**. No inter-block gaps. `HeaderSize` tells the consumer where blocks begin. Figure 1 shows the high-level structure.

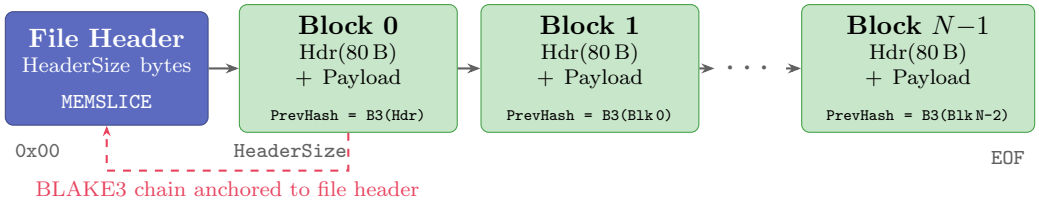

Figure 1: MSL file structure: file header followed by BLAKE3-chained blocks.

Block 0's `PrevHash` = $BLAKE3$(File Header). Each subsequent block hashes its predecessor. Consumers detect end-of-file when fewer than 80 bytes remain or the next 4 bytes do not match the block magic (`0x4D534C43`). If `BlockCount` in the file header is non-zero, consumers **SHOULD** additionally verify that the parsed block count matches the declared value.

2.1 Acquisition Modes

The file header's `Flags` field (offset `0x0C`) determines the acquisition mode and whether encryption is active:

Analysis Mode (`Investigation=0`)
The default. For research and developing new forensic approaches. Header is 64 bytes. No system-wide context required. Figure 2.

Investigation Mode (`Investigation=1`)
For forensic investigations. Block 2 **MUST** be a System Context block (`0x0050`). Section 6. Figure 3.

Either mode **MAY** set the `Encrypted` flag (bit 2), activating full-container Authenticated Encryption with Associated Data (AEAD) as defined in Section 10. When set, `HeaderSize` is 128 and the entire block stream is encrypted. Table 1 summarizes the combinations.

Table 1: Acquisition mode and encryption flag combinations.

Investigation	Encrypted	HeaderSize	Block 2
0	0	64	Memory Region or other capture-time block
0	1	128	Memory Region or other (encrypted)
1	0	64	System Context (`0x0050`, **REQUIRED**)
1	1	128	System Context (`0x0050`, **REQUIRED**, encrypted)

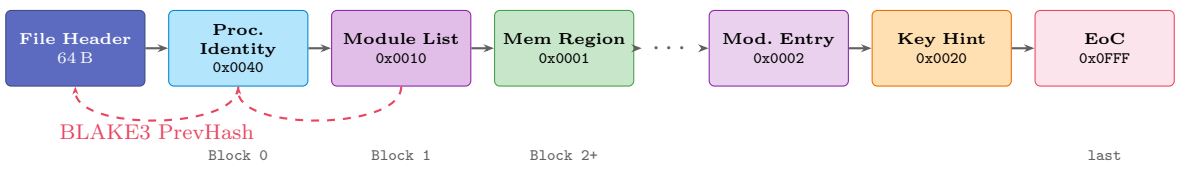

Figure 2: Analysis mode (`Investigation`=0, `Encrypted`=0).

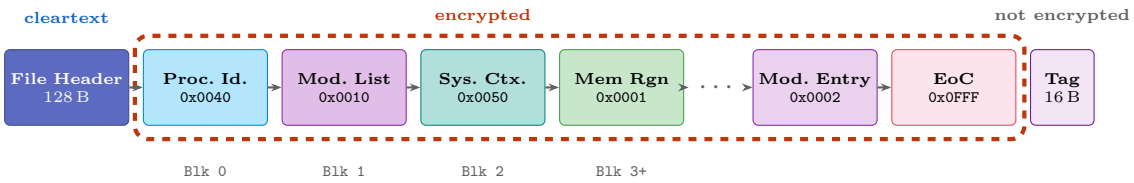

Figure 3: Investigation mode with encryption (`Investigation`=1, `Encrypted`=1). The 16-byte authentication tag is an *output* of the AEAD, not itself encrypted.

2.2 Design Principles

1. **Integrity Verification.** Append-only policy with BLAKE3 hash chain.
2. **Epistemic Honesty.** Three-state pages: Captured vs. Failed vs. Unmapped.
3. **Self-Describing.** Version, header size, OS, arch, PID, capabilities.
4. **Dual-Layer OS Abstraction.** Normalized fields + OS-native opaque blob.
5. **Dual Acquisition Modes.** Analysis mode for research; Investigation mode for forensic investigations.
6. **Confidentiality.** Full-container AEAD encryption with post-quantum key encapsulation.

Capture-time blocks are optimized for streaming, append-only acquisition. The Structural namespace (`0x1000`–`0x1FFF`) is reserved for analysis tools to add post-acquisition indexes—including seek tables and virtual address maps—that enable efficient random access without modifying the capture-time block stream.

2.3 Scope of the Integrity Guarantee

The hash chain provides *tamper detection*, not *tamper resistance*. It protects against **accidental corruption** and detects **naive modification**, including block reordering, insertion, and deletion—threats that a per-block checksum alone cannot detect, since the chain cryptographically binds each block to its predecessor and ultimately to the file header. For tamper evidence admissible in legal contexts, producers **SHOULD** complement the hash chain with external mechanisms (digital signature over `FileHash`, RFC 3161 timestamp, write-once media). When encryption is active, the AEAD tag provides stronger integrity. The hash chain alone **MUST NOT** be represented as providing forensic-grade tamper evidence.

Table 2: Integrity mechanisms: guarantees and limitations.

Mechanism	Protects Against	Limitations	Key?
BLAKE3 chain	Accidental corruption, naive modification	Attacker can recompute chain after modification	No
EoC `FileHash`	Truncation, missing blocks	Recomputable by attacker; only verifiable after decryption if encrypted	No
AEAD tag	Any tampering of encrypted block stream	Header remains cleartext (metadata leakage); requires key compromise to forge	Yes
External hash	Complete file substitution, transport corruption	Only as trustworthy as the storage of the hash itself	No

3 File Header

3.1 Base Header (64 bytes)

Table 3: File header (64 bytes minimum; 128 when `Encrypted`).

Offset	Size	Field	Abbr	Description
0x00	8	Magic	—	0x4D454D534C494345 ("MEMSLICE").
0x08	1	Endianness	E	v1.1: **MUST** be 0x01 (LE). 0x02 (BE) reserved for future versions. Other values **MUST** cause rejection.
0x09	1	HeaderSize	HS	0x40 (64) for unencrypted; 0x80 (128) when `Encrypted`. First block (or Key Encapsulation Mechanism ciphertext) starts here.
0x0A	2	Version	Ver	uint16. Major in high byte, minor in low: v1.1 = 0x0101.
0x0C	4	Flags	—	See Table 4.
0x10	8	CapBitmap	CapBm	Capability bitmap (Section 9).
0x18	16	DumpUUID	—	UUIDv4 uniquely identifying this dump.
0x28	8	Timestamp	Ts	Acquisition start (UTC, ns since epoch).
0x30	2	OSType	OS	Target OS (Table 6a).
0x32	2	ArchType	Ar	Target CPU (Table 6b).
0x34	4	PID	—	Process ID at acquisition.
0x38	1	ClockSource	CS	0x00=Unknown, 0x01=CLOCK_REALTIME, 0x02=CLOCK_MONOTONIC_RAW, 0x03=QPC, 0x04=mach_absolute_time. 0xFF=Other.
0x39	4	BlockCount	BCt	Total number of blocks (incl. EoC). 0x00000000 = unknown (streaming). Consumers **SHOULD** verify if non-zero.
0x3D	1	HashAlgo	HA	Integrity hash algorithm selector (Table 12). 0x00=BLAKE3 (default).
0x3E	2	Reserved	Rsv	**MUST** be zero.

Size: 8+1+1+2+4+8+16+8+2+2+4+1+4+1+2 = 64.

BlockCount semantics

A producer that writes blocks in streaming fashion (i.e., the total count is unknown at file creation time) **MUST** write 0x00000000. Such a producer **MAY** seek back after writing the EoC block and patch `BlockCount` to the actual value. When `BlockCount` is non-zero and the number of successfully parsed blocks differs, the file **SHOULD** be considered truncated

or corrupt.

Table 4: File header flags (`Flags`, 32 bits at offset `0x0C`).

Bit	Name	Description
0	`Imported`	File created by importing a raw dump (Section 11).
1	`Investigation`	Forensic investigation. System Context block (`0x0050`) **MUST** be Block 2 (Section 6).
2	`Encrypted`	Full-container AEAD encryption. `HeaderSize` **MUST** be 128 (Section 10).
3	`Redacted`	One or more fields have been intentionally stripped or pseudonymized (Section 6.7).
4–31	Reserved	Zero.

When `Investigation` is set and `Encrypted` is not, confidentiality **SHOULD** be ensured by external means (e.g., full-disk encryption, physically secured storage). Unencrypted Investigation mode is appropriate when the goal is rapid triage or when organizational policy provides confidentiality at the storage layer.

3.2 Encryption Extension Header (`0x40`–`0x7F`)

When `Encrypted` is set, `HeaderSize` **MUST** be 128. Bytes `0x00`–`0x3F` retain the base layout; bytes `0x40`–`0x7F` carry encryption parameters (Table 5).

Table 5: Encryption extension header (64 bytes at `0x40`–`0x7F`).

Off	Size	Field	Abbr	Description
`0x40`	1	`EncAlgo`	EA	AEAD cipher suite (Table 30).
`0x41`	1	`KDFType`	KD	`0x00`=None, `0x01`=Argon2id.
`0x42`	1	`KeyEncap`	KE	Key encapsulation (Table 31). `0x00`=None.
`0x43`	1	`Reserved`	R	Zero.
`0x44`	4	`KDFTime`	KTm	Argon2id time cost. 0 if N/A.
`0x48`	4	`KDFMemory`	KMm	Argon2id memory (KiB). 0 if N/A.
`0x4C`	1	`KDFLanes`	L	Argon2id parallelism. 0 if N/A.
`0x4D`	1	`Reserved2`	R	Zero.
`0x4E`	2	`KEMCtLen`	KCL	KEM ciphertext length. 0 when `KeyEncap`=`0x00`.
`0x50`	24	`Nonce`	—	AEAD nonce. AES-256-GCM: 12B used, rest zero. XChaCha20-Poly1305: all 24B.
`0x68`	16	`KDFSalt`	—	KDF salt. Zeros if N/A.
`0x78`	8	`Reserved3`	Rsv	Zero.

Size: 1+1+1+1+4+4+1+1+2+24+16+8 = 64. Total header: 64+64 = 128.
The extension carries all parameters needed to decrypt the block stream: the cipher suite, KDF configuration (for passphrase-derived keys), Key Encapsulation Mechanism (KEM) ciphertext

length, the AEAD nonce, and the KDF salt. Figure 4 shows the combined 128-byte header layout with base and extension regions.

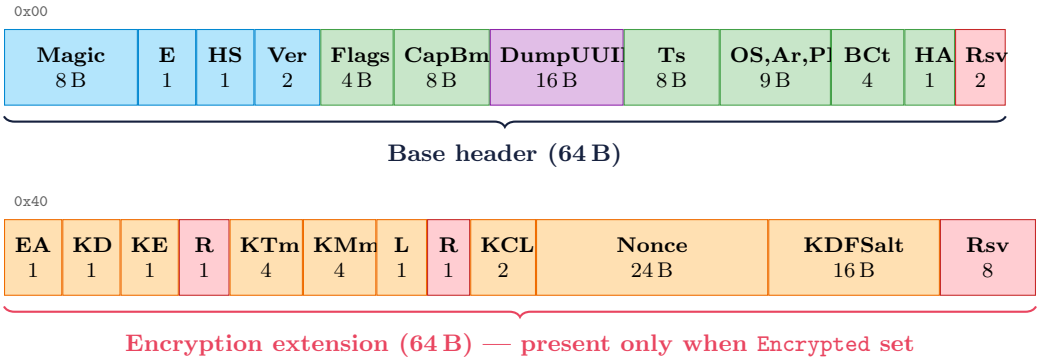

Figure 4: Extended file header (128 bytes). Abbreviations per Tables 3 and 5.

3.3 Byte Order

All multi-byte structural integers are little-endian (`Endianness=0x01`). Memory content within `PageData` is stored in the target architecture's native order. See Section 1.4 for the complete byte-order domain summary.

3.4 Version Compatibility

Unknown major version: **SHOULD** reject. Unknown minor of known major: **MAY** parse. Always use `HeaderSize` to locate Block 0.

Table 6: Target platform codes: (a) OS and (b) CPU.

(a) OS type codes (`OSType`)

Code	OS
0x0000	Windows
0x0001	Linux
0x0002	macOS
0x0003	Android
0x0004	iOS/iPadOS
0x0005	FreeBSD
0x0006	NetBSD
0x0007	OpenBSD
0x0008	QNX
0x0009	Fuchsia
0x000A--0x00FF	*Reserved*
0x0100--0xFFFE	*Vendor*
0xFFFF	Unknown

(b) Architecture codes (`ArchType`)

Code	Architecture
0x0000	x86 (IA-32)
0x0001	x86_64 (AMD64)
0x0002	ARM64 (AArch64)
0x0003	ARM32
0x0004	MIPS32
0x0005	MIPS64
0x0006	RISC-V RV32
0x0007	RISC-V RV64
0x0008	PPC32
0x0009	PPC64
0x000A	s390x
0x000B	LoongArch64
0x000C--0x00FF	*Reserved*
0x0100--0xFFFE	*Vendor*
0xFFFF	Unknown

4 Block Architecture

Every block consists of a **fixed 80-byte Common Block Header** followed by a **variable-length payload**. `BlockLength` specifies the total **on-disk** size of the block (header + payload, in compressed form if applicable). Consumers **MUST** verify `BlockLength` ≥ 80 and **MUST** reject blocks that violate this constraint. Unknown types are skipped via `BlockLength`.

4.1 Common Block Header (80 bytes)

Every block begins with the same 80-byte header, which provides type identification, length framing, UUID-based identity and parentage, and the BLAKE3 integrity chain anchor. The fixed header size ensures that consumers can always parse block boundaries without understanding payload formats. Table 7 defines the fields; Figure 5 shows the layout.

Table 7: Block header (80 bytes). All fields ≥ 8 B are 8-byte aligned.

Off	Size	Field	Abbr	Description
0x00	4	Magic	—	0x4D534C43 ("MSLC").
0x04	2	BlockType	Type	Payload selector (Section 4.3).
0x06	2	Flags	—	Per-block flags (Table 8).
0x08	4	BlockLength	Len	Total on-disk size (hdr+payload). ≥ 80.
0x0C	2	PayloadVersion	PVer	Default: 0x0001.
0x0E	2	Reserved	R	Zero.
0x10	16	BlockUUID	BlkUUID	UUIDv4. 8-byte aligned.
0x20	16	ParentUUID	ParUUID	Parent or zeros. 8-byte aligned.
0x30	32	PrevHash	—	BLAKE3 of preceding element. Zeros when encrypted. 8-byte aligned.
0x50	var	Payload	—	BlockLength − 80 bytes.

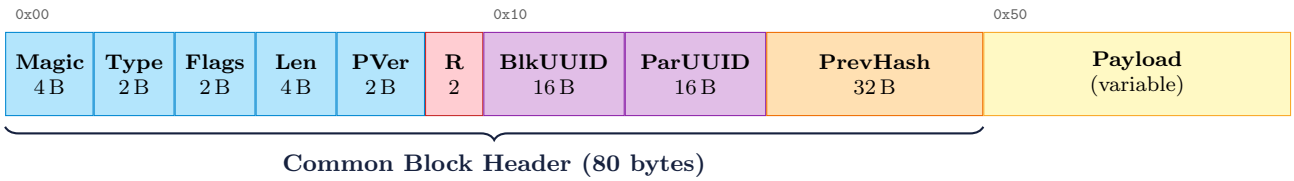

Figure 5: Common Block Header layout. Abbreviations per Table 7.

4.2 Block Flags

Table 8: Block flags (16-bit).

Bit(s)	Name	Description
0	`Compressed`	Algorithm in bits 1–2.
1–2	`CompAlgo`	00=none, 01=zstd, 10=lz4, 11=reserved.
3	`HasKeyHints`	Key Hint blocks reference this block.
4	`HasChildren`	Referenced via `ParentUUID`.
5	`Continuation`	Continuation fragment.
6–15	Reserved	Zero.

4.2.1 Compression Semantics

When the `Compressed` flag is set, only the payload portion (bytes after the 80-byte Common Block Header) is compressed. The on-disk compressed payload is structured as follows:

Table 9: Compressed payload framing.

Off	Size	Field	Description
`+0x00`	`8`	`UncompressedSize`	`uint64`. Exact size of the decompressed payload in bytes.
`+0x08`	`var`	`CompressedData`	Compressed byte stream (zstd or lz4).

`BlockLength` reflects the total on-disk size: 80+8+len(`CompressedData`). After decompression, the resulting `UncompressedSize` bytes are parsed according to the block type's payload definition. The block header (including `BlockType`, `Flags`, and `BlockLength`) is never compressed. The BLAKE3 hash chain covers the on-disk (compressed) bytes.

4.3 Block Type Registry

Table 10: Block type namespaces.

Range	Namespace	Writer	Examples
`0x0000`–`0x0FFF`	Capture-Time	Acquirer/Importer	Regions, modules, keys, system context
`0x1000`–`0x1FFF`	Structural	Analysis tool	VAS map, pointer graph
`0x2000`–`0xFFFF`	Reserved	—	Future versions

Table 11: Defined block types.

Type	Name	Description
0x0000	*Invalid*	**MUST NOT** appear.
0x0001	Memory Region	Per-page memory (Sec. 5.1).
0x0002	Module Entry	Module metadata (Sec. 5.2).
0x0010	Module List Index	Module manifest (Sec. 5.3). **MUST** be Block 1.
0x0011	Thread Context	Register state.
0x0012	File Descriptor	Open handle.
0x0013	Network Connection	Socket attribution.
0x0014	Environment Block	Env vars.
0x0015	Security Token	Credentials.
0x0020	Key Hint	Crypto key annotation (Sec. 5.6).
0x0030	Import Provenance	Raw dump metadata (Sec. 11).
0x0040	Process Identity	Exe path, cmdline, PPID (Sec. 5.4).
0x0041	Related Dump	Cross-file reference (Sec. 5.5).
0x0050	System Context	Investigation metadata (Sec. 6.2). Block 2 when Investigation set.
0x0051	Process Table	System-wide process list (Sec. 6.3).
0x0052	Connection Table	System-wide sockets (Sec. 6.4).
0x0053	Handle Table	System-wide handles (Sec. 6.5).
0x0FFF	End-of-Capture	Completeness marker (Sec. 4.5).
0x1001	VAS Map	Reconstructed virtual address space.
0x1003	Pointer Graph	Pointer relationships.

Block types listed without a section reference (e.g., 0x0011–0x0015, 0x1001, 0x1003) are reserved type codes whose payload formats will be defined in future versions of this specification. Producers **MUST NOT** emit these types until their payloads are specified. Consumers **MUST** skip them via BlockLength.

4.4 Integrity Chain (PrevHash)

The integrity hash algorithm is selected by the HashAlgo field in the file header (Table 3). All integrity fields within a single file—PrevHash, EoC FileHash, and Module Entry DiskHash—**MUST** use the same algorithm; producers **MUST NOT** mix algorithms within a file. All registered algorithms produce a 32-byte output, so field layouts are independent of the selection.

Let $H(\cdot)$ denote the selected algorithm. Then:

- **Block 0:** PrevHash $= H($File Header, all HeaderSize bytes$)$.
- **Block** i **(**$i \geq 1$**):** PrevHash $= H(\text{Block}_{i-1})$.

Table 12: Integrity hash algorithm codes (`HashAlgo`).

Code	Algorithm	Output	Notes
`0x00`	BLAKE3 [4]	32 B	**Default.** High throughput via tree hashing and SIMD.
`0x01`	SHA-256 [5]	32 B	FIPS 180-4. Hardware-accelerated on Intel SHA-NI, AMD Zen, and ARMv8 Cryptographic Extension.
`0x02`	SHA-512/256 [5]	32 B	FIPS 180-4. Truncated SHA-512 with distinct initial values; not length-extension equivalent to SHA-512. Often faster than SHA-256 on 64-bit platforms lacking SHA-NI.
`0x03--0xFE`	*Reserved*	—	Reserved for future algorithms.
`0xFF`	Other	32 B	Non-standard; interpretation out of scope.

Default and rationale. `HashAlgo=0x00` (BLAKE3) is **RECOMMENDED** for general use and is the default for all implementations. Producers **SHOULD** select `0x01` (SHA-256) or `0x02` (SHA-512/256) only when FIPS 140 compliance, ecosystem interoperability, or organizational policy requires a NIST-standardized hash; or when platform measurements indicate a NIST hash outperforms BLAKE3—for example, aarch64 systems with the ARMv8 Cryptographic Extension, where SHA-256 hardware acceleration can outperform software BLAKE3.

Scope of selection. The `HashAlgo` selection applies only to integrity hashing: `PrevHash`, `FileHash`, and `DiskHash`. The key derivation function defined in Section 10.4 always uses HKDF-BLAKE3 regardless of `HashAlgo`; integrity hashing and key derivation are independent design choices and are not configured together.

Interpretation of references. Where this specification refers to "BLAKE3" in integrity contexts—`PrevHash` (this section), `FileHash` (Section 4.5), and `DiskHash` (Section 5.2)—the reference denotes the algorithm selected by `HashAlgo`. References to HKDF-BLAKE3 (Section 10.4) are not subject to `HashAlgo` and always denote BLAKE3.

Scope of hashing. Hashes cover raw on-disk bytes (compressed form if applicable). When `Encrypted` is set, all `PrevHash` fields **MUST** be zero (Section 10.6).

4.5 End-of-Capture Block (`0x0FFF`)

Completeness marker. An acquirer **SHOULD** emit it as the final capture-time block. Consumers **MUST** treat any bytes following a verified EoC block as non-MSL trailing data.

Table 13: End-of-Capture payload.

Off	Size	Field	Description
`+0x00`	32	`FileHash`	BLAKE3 of all bytes from offset 0 through the last byte before this EoC block. Populated even when encrypted (computed over plaintext).
`+0x20`	8	`AcqEnd`	Acquisition end time (UTC, ns since epoch).
`+0x28`	8	`Reserved`	Zero.

The next sections specify payload contents: per-process payloads (Section 5), investigation-mode payloads (Section 6), and the three-state page model (Section 7).

5 Capture-Time Payloads

Payload data begins at byte `0x50`. All offsets below are **relative to the payload start**.

5.1 Memory Region (`0x0001`)

Table 14: Memory Region payload.

Off	Size	Field	Abbr	Description
+0x00	8	BaseAddr	—	Virtual address of first byte.
+0x08	8	RegionSize	RegnSz	**MUST** be a multiple of PageSize.
+0x10	1	Protection	Pr	Bit 0=R, 1=W, 2=X, 3=Guard, 4=CoW.
+0x11	1	RegionType	RT	00=Unknown, 01=Heap, 02=Stack, 03=Image, 04=MappedFile, 05=Anonymous, 06=SharedMem, FF=Other.
+0x12	1	PageSizeLog2	PL	$2^{\texttt{PageSizeLog2}}$ bytes. Range $[10, 40]$.
+0x13	5	Reserved	R	Zero.
+0x18	8	Timestamp	Ts	Region acquisition time.
+0x20	var	PageStateMap	—	2 bits/page (Section 7). Pad to 8B.
$+m$	var	PageData	—	Captured pages only.

PageStateMap size. $P = \texttt{RegionSize} \gg \texttt{PageSizeLog2}$. Map: pad8$((P+3)/4)$ bytes.

Maximum region size and continuation blocks

The maximum payload per block is $2^{32} - 81$ bytes, derived from the 32-bit `BlockLength` (maximum value $2^{32} - 1$) minus the 80-byte Common Block Header. Regions exceeding this limit **MUST** be split into continuation blocks (`Continuation` flag set).

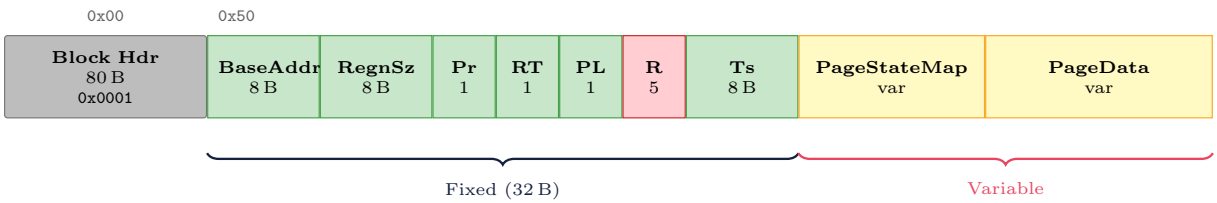

Figure 6: Memory Region (`0x0001`). Abbreviations per Table 14.

5.2 Module Entry (`0x0002`)

Stores the identity and metadata of a loaded library (DLL, .so): file path, version string, on-disk hash, and OS-native loader data. `ParentUUID` **MUST** reference the Module List Index (`0x0010`). `BlockUUID` **MUST** match the pre-assigned `ModuleUUID` from the manifest.

Table 15: Module Entry payload.

Off	Size	Field	Abbr	Description
+0x00	8	BaseAddr	—	Load address.
+0x08	8	ModuleSize	ModSz	Mapping size.
+0x10	2	PathLen	PL	Path length (incl. null, before padding).
+0x12	2	VersionLen	VL	Version length. 0 if unavailable.
+0x14	4	Reserved	R	Zero.
+0x18	var	Path	—	UTF-8, pad8.
$+p$	var	Version	—	UTF-8, pad8.
$+v$	32	DiskHash	—	BLAKE3 of on-disk binary. 32 zero bytes if unavailable.
$+v$+32	4	BlobLen	BL	NativeBlob size.
$+v$+36	4	Reserved2	R	Zero.
$+v$+40	var	NativeBlob	—	OS-native opaque data.

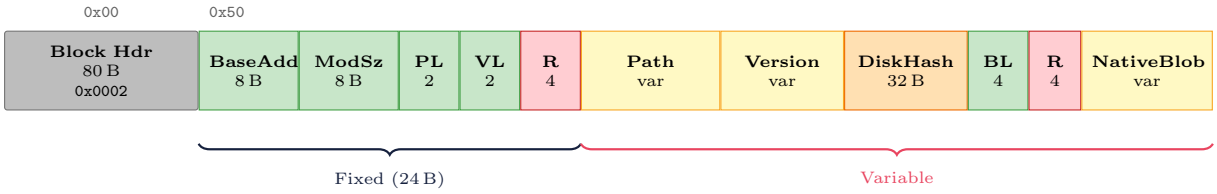

Figure 7: Module Entry (0x0002). Abbreviations per Table 15.

5.3 Module List Index (0x0010)

MUST be Block 1. Pre-generates UUIDv4 for each Module Entry block.

Table 16: Module List Index payload.

Off	Size	Field	Abbr	Description
— Preamble —				
+0x00	4	EntryCount	ECt	Number of module entries.
+0x04	4	Reserved	R	Zero.
— Entry (repeated ×EntryCount), offsets relative to entry start —				
+0x00	16	ModuleUUID	ModUUID	Pre-assigned BlockUUID.
+0x10	8	BaseAddr	—	Module load address.
+0x18	8	ModuleSize	ModSz	Mapping size.
+0x20	2	PathLen	PL	Path length (incl. null).
+0x22	2	Reserved	R	Zero.
+0x24	4	Reserved2	R	Zero.
+0x28	var	Path	—	UTF-8, pad8.

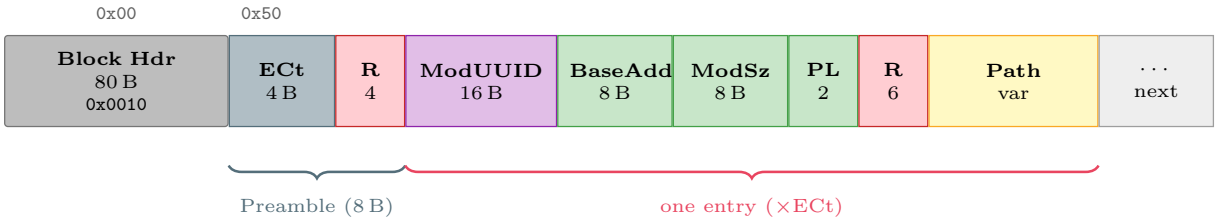

Figure 8: Module List Index (0x0010). Abbreviations per Table 16.

5.4 Process Identity (0x0040)

MUST be Block 0 for live acquisition (or after Import Provenance for imports).

Table 17: Process Identity payload.

Off	Size	Field	Abbr	Description
+0x00	4	PPID	—	Parent PID. 0 if unknown.
+0x04	4	SessionID	SID	0 if unknown.
+0x08	8	StartTime	StartTm	Process start time. 0 if unknown.
+0x10	2	ExePathLen	EPL	Incl. null.
+0x12	2	CmdLineLen	CLL	0 if unavailable.
+0x14	4	Reserved	R	Zero.
+0x18	var	ExePath	—	UTF-8, pad8.
$+e$	var	CmdLine	—	UTF-8, pad8.

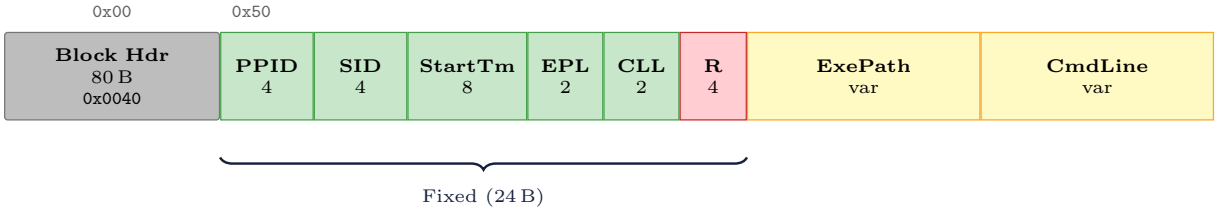

Figure 9: Process Identity (0x0040). Abbreviations per Table 17.

5.5 Related Dump (0x0041)

An acquirer **MAY** emit one or more Related Dump blocks to record relationships between concurrently captured processes (parent/child, shared-memory peers, IPC partners).

Table 18: Related Dump payload (24 bytes, fixed).

Off	Size	Field	Abbr	Description
+0x00	16	RelatedDumpUUID	RelDumpUUID	DumpUUID of related file.
+0x10	4	RelatedPID	RelPID	0 if unknown.
+0x14	2	Relationship	Rel	0x0001=Parent, 0x0002=Child, 0x0003=SharedMemory, 0x0004=IPC peer, 0x0005=Thread group, 0xFFFF=Other.
+0x16	2	Reserved	R	Zero.

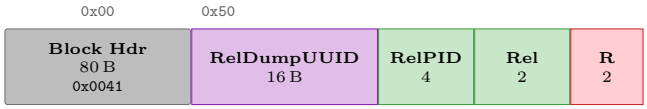

Figure 10: Related Dump (0x0041). Abbreviations per Table 18.

5.6 Key Hint (0x0020)

A Key Hint block annotates a region of captured memory that is believed to contain cryptographic key material. It records the key's location (via RegionUUID and byte offset), its type and protocol association, and a confidence level reflecting how the key was identified (speculative scan, heuristic match, or confirmed via instrumentation). Key Hints are **OPTIONAL**; their presence is indicated by the CryptoHints bit in CapBitmap and the HasKeyHints flag on the referenced Memory Region block.

Table 19: Key Hint payload.

Off	Size	Field	Abbr	Description
+0x00	16	RegionUUID	RgnUUID	Memory Region BlockUUID.
+0x10	8	RegionOffset	RgnOff	Byte offset in PageData.
+0x18	4	KeyLen	KLen	0 if unknown.
+0x1C	2	KeyType	KT	Table 20.
+0x1E	2	Protocol	Pr	Table 20.
+0x20	1	Confidence	C	0x00=Speculative, 0x01=Heuristic, 0x02=Confirmed.
+0x21	1	KeyState	S	0x00=Unknown, 0x01=Active, 0x02=Expired.
+0x22	2	Reserved	R	Zero.
+0x24	4	NoteLen	NL	0 if none.
+0x28	4	Reserved2	R	Zero.
+0x2C	var	Note	—	UTF-8, pad8.

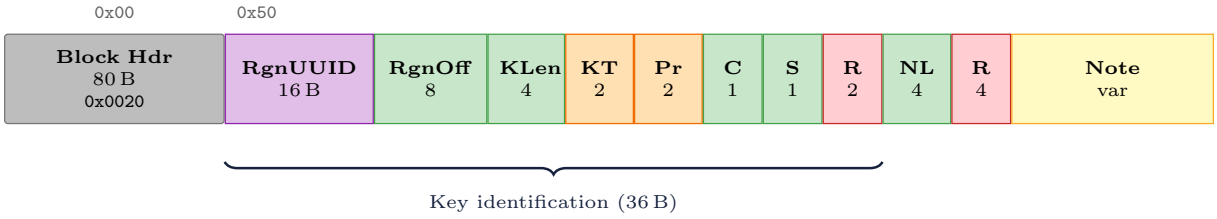

Figure 11: Key Hint (0x0020). Abbreviations per Table 19.

Table 20: Key type and protocol codes.

Key type codes (`KeyType`)

Code	Type
0x0000	Unknown
0x0001	Pre-Master Secret
0x0002	Master Secret
0x0003	Session Key
0x0004	Handshake Secret
0x0005	App Traffic Secret
0x0006	RSA Private Key
0x0007	ECDH Private Key
0x0008	IKE SA Key
0x0009	ESP/AH Key
0x000A	SSH Session Key
0x000B	WireGuard Key
0x000C	ML-KEM Private Key
0xFFFF	Other

Protocol codes (`Protocol`)

Code	Protocol
0x0000	Unknown
0x0001	TLS 1.2
0x0002	TLS 1.3
0x0003	DTLS 1.2
0x0004	DTLS 1.3
0x0005	QUIC
0x0006	IKEv2/IPsec
0x0007	SSH
0x0008	WireGuard
0x0009	PQ-TLS (hybrid)
0xFFFF	Other

6 Investigation Mode

When `Investigation` (bit 1) is set, the acquirer **MUST** capture system-wide context alongside per-process memory: acquisition operator, system hostname, and system-wide tables (process list, network connections, open handles). When not set, the acquisition is in Analysis mode and **MUST** follow Figure 2.

6.1 Block Ordering

1. **Block 0:** Process Identity (`0x0040`).
2. **Block 1:** Module List Index (`0x0010`).
3. **Block 2:** System Context (`0x0050`) — **REQUIRED**.

System-wide table blocks (`0x0051`–`0x0053`) follow with `ParentUUID` referencing the System Context block.

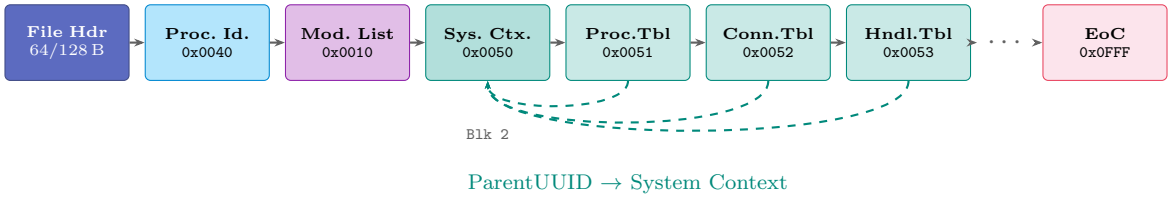

Figure 12: Investigation mode block sequence. System table blocks reference System Context via `ParentUUID`.

6.2 System Context Block (`0x0050`)

The System Context block captures system-wide metadata at acquisition time: the identity of the acquisition operator, the target system's hostname and OS version, and a bitmap indicating which system-wide table blocks (`0x0051`–`0x0053`) are present. This block **MUST** be Block 2 when `Investigation` is set. The `TableBitmap` allows consumers to determine which tables to expect without scanning ahead.

Table 21: System Context payload.

Off	Size	Field	Abbr	Description
+0x00	8	BootTime	BootTm	System boot time. 0 if unknown.
+0x08	4	TargetCount	TCt	Processes targeted this session.
+0x0C	4	TableBitmap	TBm	Bit 0=ProcessTable, 1=ConnectionTable, 2=HandleTable.
+0x10	2	AcqUserLen	—	Incl. null.
+0x12	2	HostnameLen	—	Incl. null.
+0x14	2	DomainLen	—	0 if N/A.
+0x16	2	OSDetailLen	—	
+0x18	2	CaseRefLen	—	0 if none.
+0x1A	6	Reserved	R	Zero.
+0x20	var	AcqUser	—	UTF-8, pad8. Acquisition operator.
$+u$	var	Hostname	—	UTF-8, pad8.
$+h$	var	Domain	—	UTF-8, pad8. Omitted if len=0.
$+d$	var	OSDetail	OSDet	UTF-8, pad8.
$+o$	var	CaseRef	—	UTF-8, pad8. Omitted if len=0.

The TableBitmap field advertises which tables and system-context extension blocks are present in the slice. Each bit is associated with a single block identifier, so that a reader can derive the expected block inventory directly from the SystemContext block rather than by scanning the slice. Bit assignments are listed in Table 22; bits 10–31 are reserved and shall be emitted as zero.

Table 22: TableBitmap bit assignments.

Bit	Mask	Block	Set when
0	0x0001	ProcessTable (0x0051)	The system process table is emitted.
1	0x0002	ConnectionTable (0x0052)	The system socket-connection table is emitted.
2	0x0004	HandleTable (0x0053)	The per-target handle table is emitted.
3	0x0008	ConnectivityTable (0x0054)	The collector produced at least one row of kernel network state (routes, ARP, packet sockets, interface counters, socket-family aggregates, or MIB counters).

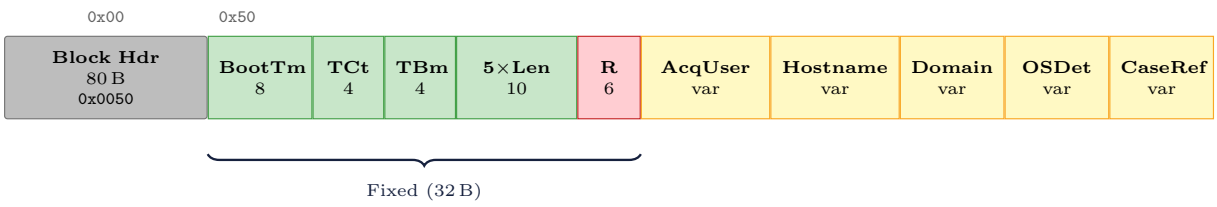

Figure 13: System Context (0x0050). Abbreviations per Table 21.

The block definitions that follow specify the system-context metadata classes in the identifier range `0x0051`–`0x0054`. Each block is additive: readers that do not recognize a block type skip it according to the rules of Section 4, and the format therefore remains forward compatible. Presence is advertised through a dedicated bit in the SystemContext `TableBitmap` (Table 22).

6.3 Process Table (`0x0051`)

System-wide process snapshot. `ParentUUID` **MUST** reference the System Context block.

Table 23: Process Table payload.

Off	Size	Field	Abbr	Description
— Preamble —				
+0x00	4	EntryCount	ECt	Number of process entries.
+0x04	4	Reserved	R	Zero.
— Entry (repeated ×EntryCount), offsets relative to entry start —				
+0x00	4	PID	—	
+0x04	4	PPID	—	Parent PID.
+0x08	4	UID	—	POSIX UID or SID hash.
+0x0C	1	IsTarget	T	0x01 if acquisition target.
+0x0D	3	Reserved	R	Zero.
+0x10	8	StartTime	StTm	0 if unknown.
+0x18	8	RSS	—	0 if unknown.
+0x20	2	ExeNameLen	—	Incl. null.
+0x22	2	CmdLineLen	—	0 if unavailable.
+0x24	2	UserLen	—	0 if unavailable.
+0x26	2	Reserved2	R	Zero.
+0x28	var	ExeName	—	UTF-8, pad8.
+e	var	CmdLine	—	UTF-8, pad8.
+c	var	User	—	UTF-8, pad8.

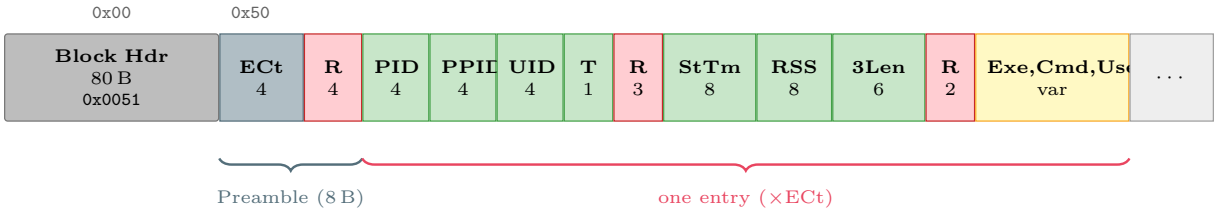

Figure 14: Process Table (`0x0051`). Abbreviations per Table 23.

6.4 Connection Table (`0x0052`)

System-wide network sockets. `ParentUUID` **MUST** reference System Context.

Port byte order

`LocalPort` and `RemotePort` are structural integers and therefore stored as little-endian `uint16`, regardless of the network byte order (big-endian) used by OS socket APIs. Producers **MUST** convert from network byte order before writing.

Table 24: Connection Table payload.

Off	Size	Field	Abbr	Description
— Preamble —				
+0x00	4	EntryCount	ECt	Number of connection entries.
+0x04	4	Reserved	R	Zero.
— Entry (repeated ×EntryCount), 48 bytes fixed, offsets relative to entry start —				
+0x00	4	PID	—	0 if unknown.
+0x04	1	Family	F	0x02=IPv4, 0x0A=IPv6.
+0x05	1	Protocol	P	0x06=TCP, 0x11=UDP.
+0x06	1	State	S	TCP state. 0x00=N/A.
+0x07	1	Reserved	R	Zero.
+0x08	16	LocalAddr	—	IPv4: 4B used, rest zero.
+0x18	2	LocalPort	LP	Little-endian uint16.
+0x1A	2	Reserved2	R	Zero.
+0x1C	16	RemoteAddr	—	Zeros for LISTEN.
+0x2C	2	RemotePort	RP	0 for LISTEN/UDP.
+0x2E	2	Reserved3	R	Zero.

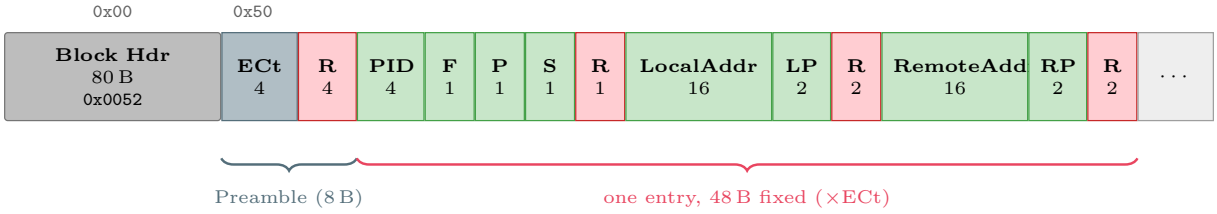

Figure 15: Connection Table (0x0052). Abbreviations per Table 24.

6.5 Handle Table (0x0053)

System-wide file descriptors/handles. ParentUUID **MUST** reference System Context.

Table 25: Handle Table payload.

Off	Size	Field	Abbr	Description
— Preamble —				
+0x00	4	EntryCount	ECt	Number of handle entries.
+0x04	4	Reserved	R	Zero.
— Entry (repeated ×EntryCount), offsets relative to entry start —				
+0x00	4	PID	—	
+0x04	4	FD	—	Descriptor/handle number.
+0x08	1	HandleType	HT	0x00=Unknown, 0x01=File, 0x02=Dir, 0x03=Socket, 0x04=Pipe, 0x05=Device, 0x06=Registry, 0xFF=Other.
+0x09	1	Reserved	R	Zero.
+0x0A	2	PathLen	PL	0 if unavailable.
+0x0C	4	Reserved2	R	Zero.
+0x10	var	Path	—	UTF-8, pad8.

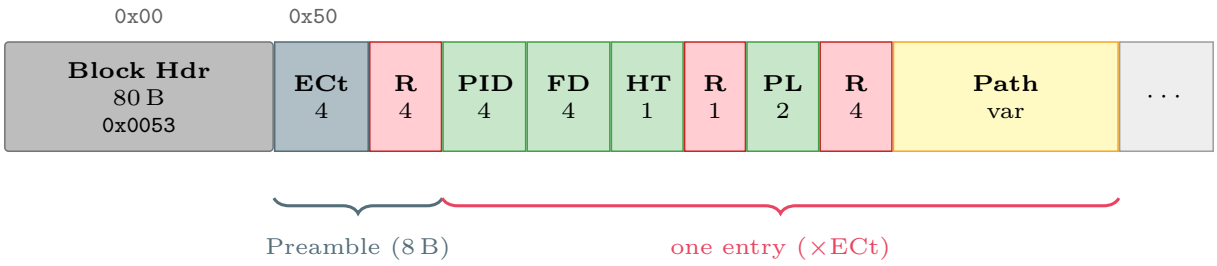

Figure 16: Handle Table (0x0053). Abbreviations per Table 25.

6.6 Connectivity Table Block (0x0055)

The Connectivity Table records kernel network state that has no socket-endpoint representation and therefore does not fit the schema of the Connection Table block (0x0052). The two blocks are complementary: endpoints with process attribution remain in the Connection Table, while routing entries, neighbour-discovery (ARP) caches, raw packet sockets, per-interface counters, aggregate socket-family statistics, and Management Information Base (MIB) counters are recorded here. The separation allows downstream analyzers to distinguish the two forensic questions of *who* was connected to *what* and *what was the network state at capture time*. The latter is essential for detecting raw-socket observers whose presence is invisible to the conventional socket table, because they possess neither an address nor a port tuple.

The payload is a heterogeneous tagged-row table. Each row begins with a one-byte row-type discriminator, followed by a two-byte row length and a row-type-specific body. Seven row shapes are defined; readers that do not recognize a row type advance by RowLen bytes.

Table 26: Connectivity Table payload.

Off	Size	Field	Abbr	Description
+0x00	4	RowCount	RC	Total number of rows across all row types.
+0x04	4	Reserved	—	Must be zero.
+0x08	var	Rows	—	Concatenation of tagged rows (see Table 27).

Each row is prefixed by `RowType(1)` `RowLen(2)` and followed by `RowLen` bytes of row-specific body. The defined row types are enumerated in Table 27. String-typed fields within row bodies use a two-byte length prefix followed by `Len` bytes of UTF-8 content. All multi-byte integers are little-endian. IPv4 addresses are carried as four raw bytes in network order; IPv6 addresses as sixteen raw bytes.

Table 27: Connectivity Table row-type discriminators.

Type	Name	Body schema
0x01	IPv4 route	`IfaceLen(2), Iface, Dest(4), Gateway(4), Mask(4), Flags(2), Metric(4), Mtu(4)`
0x02	IPv6 route	`IfaceLen(2), Iface, Dest(16), DestPrefix(1), NextHop(16), Metric(4), Flags(4)`
0x03	ARP entry	`Family(1), Ip(4), HwType(2), Flags(2), HwAddr(6), IfaceLen(2), Iface`
0x04	Packet socket	`Pid(4), Inode(8), Proto(2), IfaceIndex(4), User(4), Rmem(8)`
0x05	Interface statistics	`IfaceLen(2), Iface, RxBytes(8), RxPkts(8), RxErr(8), RxDrop(8), TxBytes(8), TxPkts(8), TxErr(8), TxDrop(8)`
0x06	Socket-family aggregate	`Family(1), InUse(4), Alloc(4), Mem(8)`
0x07	MIB counter	`MibLen(2), Mib, CounterLen(2), Counter, Value(8)`

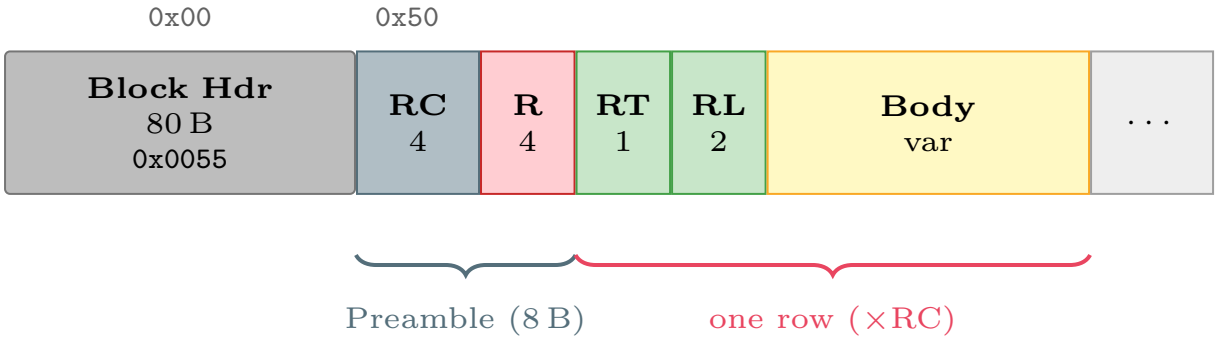

Figure 17: Connectivity Table (`0x0055`). Abbreviations per Table 26 and Table 27.

6.7 Redaction and Data Minimization

Investigation-mode fields such as `AcqUser`, `Hostname`, `Domain`, and `CaseRef` may contain personally identifiable information or data subject to disclosure restrictions. This section provides guidance for producers that must balance forensic completeness with privacy requirements.

- Producers **MAY** emit zero-length strings (Len=0) for `AcqUser`, `Domain`, and `CaseRef` when

organizational policy requires data minimization. A conformant consumer **MUST** handle zero-length optional strings gracefully.

- When any field has been intentionally stripped or pseudonymized, producers **MUST** set the `Redacted` flag (bit 3) in the file header. This distinguishes intentional redaction from fields that were merely unavailable at acquisition time: a zero-length string with `Redacted`=0 means "not captured"; the same string with `Redacted`=1 means "captured but removed."
- When sharing MSL files for research or cross-organizational analysis, producers **SHOULD** strip or pseudonymize personally identifiable fields prior to distribution and set the `Redacted` flag.
- `Hostname` and `OSDetail` carry forensic value that may be essential for investigation continuity. Redaction of these fields **SHOULD** be documented in the case log.

7 Three-State Virtual Address Map

Each page within a Memory Region is classified into one of three states, encoded as 2 bits in the `PageStateMap`. This three-state model enforces *epistemic honesty*: rather than silently omitting pages that could not be read, the format distinguishes successful capture from read failures and TOCTOU (Time-of-check to time-of-use) races (pages that were mapped when the region was enumerated but absent when actually read). Table 28 defines the states; Figure 18 shows an example map.

Table 28: Page states.

Bits	State	Meaning	PageData
00	Captured	Read OK; stored.	PageSize bytes
01	Failed	Mapped but unreadable.	0
10	Unmapped	TOCTOU: absent at read time.	0
11	*Reserved*	Treat as Failed.	0

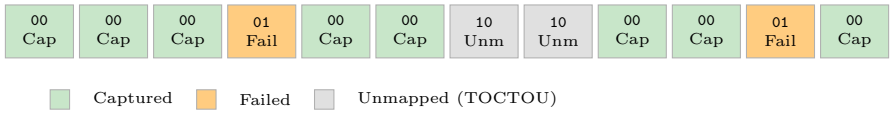

Figure 18: Example PageStateMap for a 12-page region.

8 Block Cross-Referencing

Layer 1: `ParentUUID`. Header-level hierarchy. Module Entry → Module List; system tables → System Context.

Layer 2: Payload-embedded UUIDs. Key Hint `RegionUUID` → Memory Region.

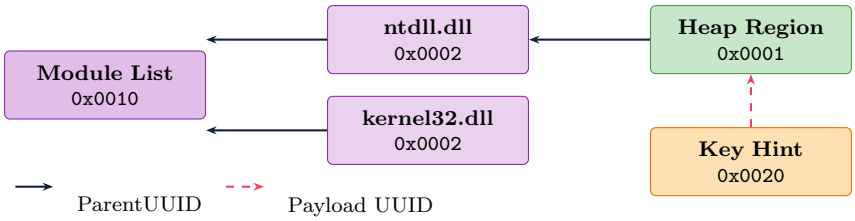

Figure 19: Block cross-referencing.

Dangling references. Consumers encountering a `ParentUUID` or payload-embedded UUID that does not match any block in the file **SHOULD** log the dangling reference and continue parsing. The block with the unresolvable reference **SHOULD** be treated as an orphan but **MUST NOT** be discarded, as the referenced block may have been lost to corruption rather than being absent by design.

9 Capability Bitmap

The 64-bit `CapBitmap` in the file header advertises which categories of data the file contains. Consumers can inspect the bitmap before parsing any blocks to determine whether the information they need (e.g., module metadata, crypto key hints, system-wide tables) was captured. Each bit corresponds to one or more block types as shown in Table 29. Producers **MUST** set bits accurately; consumers **SHOULD** use the bitmap to report missing data categories rather than silently producing incomplete results.

Table 29: Capability bits (64-bit `CapBitmap`).

Bit	Name	Description
0	MemoryRegions	`0x0001`.
1	ModuleList	`0x0010`/`0x0002`.
2	ThreadContexts	`0x0011`.
3	FileDescriptors	`0x0012`.
4	NetworkState	`0x0013`.
5	EnvironmentVars	`0x0014`.
6	SharedMemory	IPC identifiers.
7	SecurityContext	`0x0015`.
8	ProcessIdentity	`0x0040`.
9	RelatedDumps	`0x0041`.
10	CryptoHints	`0x0020`.
11	SystemContext	`0x0050`. **MUST** be set when `Investigation` set.
12	SystemProcessTable	`0x0051`.
13	SystemNetworkTable	`0x0052`.
14	SystemHandleTable	`0x0053`.
15–63	Reserved	Zero.

10 Encryption

10.1 Scope

When the `Encrypted` flag (bit 2) is set, the entire block stream is protected by AEAD (Authenticated Encryption with Associated Data). The file header remains in cleartext for format identification. All block data—including block types, UUIDs, timestamps, module paths, and memory content—is encrypted.

Full-container encryption is chosen over per-block encryption to prevent metadata leakage: per-block schemes would expose block types, block counts, block sizes, and timing patterns, all of which can reveal forensically sensitive information (e.g., which libraries are loaded, how many memory regions exist) without the decryption key.

10.2 Encrypted File Layout

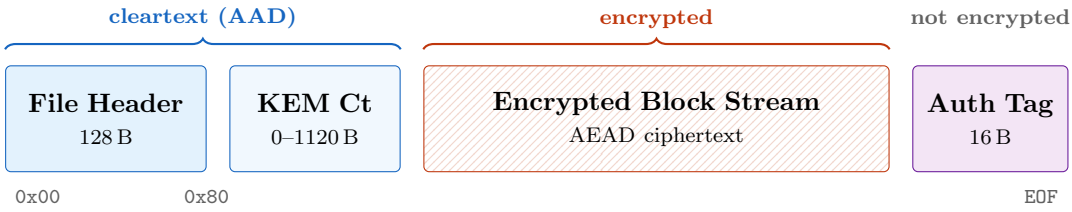

Figure 20: Encrypted file layout. The AEAD encrypts the block stream; the tag is an output, not itself encrypted.

1. **File header (128 B, cleartext):** Authenticated as Associated Data (AAD) to the AEAD, along with any KEM ciphertext. Tampering causes decryption failure.
2. **KEM ciphertext (0 or variable, cleartext):** Present when `KeyEncap`≠`0x00`. Size is `KEMCtLen`. Also in AAD.
3. **Encrypted block stream:** AEAD ciphertext of Block 0 through EoC. The only encrypted region.
4. **AEAD authentication tag (16 B, not encrypted):** Output of the AEAD. Required for decryption. Requires the symmetric key.

Chain-of-custody without the key. Producers **SHOULD** record an external hash of the complete file at acquisition time (BLAKE3 or SHA-256, offset 0 to EOF). This digest **SHOULD** be stored in the case evidence log, not in the file.

10.3 Cipher Suites

Table 30: AEAD cipher suites.

Code	Suite	Nonce	Tag	Notes
`0x01`	AES-256-GCM	12 B	16 B	Default. Fast with AES-NI.
`0x02`	XChaCha20-Poly1305	24 B	16 B	Safe random nonce.

Both ciphers provide 256-bit keys. AES-256 and ChaCha20 are post-quantum safe at the symmetric level (Grover reduces effective key strength by half: AES-256 → 128-bit equivalent). The post-quantum vulnerability lies in key *encapsulation*, addressed below.

10.4 Key Encapsulation

The `KeyEncap` field specifies how the content encryption key (CEK) was protected. After KEM decapsulation, the shared secret is passed through HKDF-BLAKE3 (salt=`DumpUUID`, info=`"MSL-CEK-v1"`) to derive the 256-bit CEK.

Table 31: Key encapsulation mechanisms.

Code	Mechanism	Ct Size	Notes
`0x00`	None	0	Raw key (passphrase or external).
`0x01`	X25519	32 B	Classical ECDH.
`0x02`	ML-KEM-768	1088 B	FIPS 203 [7]. NIST PQ standard.
`0x03`	ML-KEM-1024	1568 B	FIPS 203. Higher margin.
`0x04`	X25519+ML-KEM-768	1120 B	Hybrid. **Recommended.**
`0x05--0xFE`	*Reserved*	—	
`0xFF`	Other	var	Non-standard.

Operational model: symmetric key vs. KEM

The symmetric cipher always encrypts the data. `KeyEncap` determines how the CEK reaches the intended recipient:

- `KeyEncap=0x00` **(passphrase/raw key):** The responder provides a passphrase or externally managed key. Suitable for single-operator scenarios.
- `KeyEncap≠0x00` **(KEM):** The acquirer encrypts to the organization's public key. The responder need not know any secret. The private key is held by the analysis team or in an HSM. This provides separation of duties: the responder who captures evidence cannot decrypt it.
- **Passphrase + KEM (dual control):** Both the responder's passphrase and the organization's private key are required, preventing either party from accessing evidence alone.

The post-quantum options protect against "harvest now, decrypt later" attacks.

Hybrid X25519 + ML-KEM-768. The recommended hybrid scheme (`0x04`) concatenates shared secrets from X25519 and ML-KEM-768 before KDF, ensuring the file remains secure as long as *either* assumption holds. This concatenate-then-KDF construction follows the approach analyzed by Bindel et al. [8] and adopted by TLS 1.3 hybrid key exchange [9]. The hybrid mode is the **recommended** setting for Investigation mode captures.

10.5 Key Derivation

`0x00` — **None**
: 256-bit CEK provided externally. KDF fields are zero.

`0x01` — **Argon2id**
: Passphrase-derived. Minimum: time=3, memory=65536 KiB, lanes=4.

Because `KDFSalt` is generated uniquely per file (16 random bytes), the same passphrase produces a different CEK for each capture, eliminating nonce-reuse risk even when nonces are generated independently across files.

10.6 Interaction with the Integrity Chain

When `Encrypted` is set:

- `PrevHash`: **MUST** be zero. AEAD authenticates atomically.
- EoC `FileHash`: **SHOULD** be computed over plaintext (enables post-decryption verification).
- AAD: File header (128 B) + any KEM ciphertext.

10.7 Streaming Encryption

Producers **SHOULD** encrypt incrementally: (1) init AEAD with key, nonce, AAD; (2) for each block, serialize, encrypt, write; (3) after EoC, finalize AEAD, append 16-byte tag. A parallel BLAKE3 hasher computes plaintext FileHash before encryption. $O(1)$ memory.

11 Raw Dump Import

An MSL file may be created by importing a pre-existing raw memory dump (ELF core, Windows Minidump, ProcDump output, or plain byte stream) rather than by live acquisition. The Import Provenance block (`0x0030`) records the origin of the imported data: the source format, the tool that performed the conversion, the original file size, and an optional free-text note. When `Imported` (bit 0) is set in `Flags`, this block **MUST** appear as Block 0.

Table 32: Import Provenance payload.

Off	Size	Field	Description
+0x00	2	SourceFormat	Table 33.
+0x02	2	Reserved	Zero.
+0x04	4	ToolNameLen	Incl. null.
+0x08	8	ImportTime	ns since epoch.
+0x10	8	OrigFileSize	0 if unknown.
+0x18	4	NoteLen	0 if none.
+0x1C	4	Reserved2	Zero.
+0x20	var	ToolName	UTF-8, pad8.
$+t$	var	Note	UTF-8, pad8.

Table 33: Source format codes.

Code	Format
0x0000	Unknown
0x0001	Raw byte stream
0x0002	ELF core dump
0x0003	Windows Minidump
0x0004	macOS core dump
0x0005	ProcDump
0xFFFF	Other

Procedure: (1) Set `Flags` bit 0. (2) Block 0: Import Provenance. (3) Regions from source. (4) Modules if available. (5) Hash chain and EoC.

12 Parsing Walkthrough

Step 1:
Read 9 bytes. Verify magic. `Endianness` **MUST** be `0x01`. Read `HeaderSize`.

Step 2:
If `Encrypted`: read extension (`0x40`–`0x7F`). If `KEMCtLen`>0, read KEM ciphertext. Obtain key. Decrypt. Continue with plaintext.

Step 3:
Read blocks at `HeaderSize` (+`KEMCtLen` if encrypted). Verify `BlockLength`≥80. Parse or skip via `BlockLength`. If `BlockCount`>0, verify parsed count matches.

Step 4:
If unencrypted: verify BLAKE3 chain. If encrypted: verify AEAD tag.

Step 5:
Verify EoC `FileHash` if present.

Step 6:
`Investigation`? Expect Block 2 = System Context.

Step 7:
`Imported`? Read Import Provenance.

13 Worked Example

Module Entry block for `ntdll.dll` `v10.0.22621.1` at `0x7FFDD4D00000`. Path=30B (pad8=32). Version=13B (pad8=16). Payload=496B. Block=576 (`0x0240`).

Table 34: ntdll.dll Module Entry.

Off	Field	Hex	Value
— *Header (80B)* —			
0x00	Magic	4D 53 4C 43	MSLC
0x04	BlockType	02 00	Module Entry
0x08	BlockLength	40 02 00 00	576
0x0C	PayloadVersion	01 00	1
0x10	BlockUUID	(16B)	UUIDv4
0x20	ParentUUID	(16B)	Module List UUID
0x30	PrevHash	(32B)	BLAKE3 of prev
— *Payload (496B)* —			
0x50	BaseAddr	00 00 D0 D4 FD 7F 00 00	0x7FFDD4D00000
0x58	ModuleSize	00 00 20 00 00 00 00 00	2 MiB
0x60	PathLen	1E 00	30
0x62	VersionLen	0D 00	13
0x68	Path (32B)	43 3A 5C 57...	ntdll.dll
0x88	Version (16B)	31 30 2E 30...	10.0.22621.1
0x98	DiskHash (32B)	...	BLAKE3 of binary
0xB8	BlobLen	80 01 00 00	384
0xC0	NativeBlob	(384B)	LDR_DATA_TABLE_ENTRY

14 Conformance

14.1 Producer

A conformant producer **MUST**: (1) write valid header with `Magic`, `Endianness`=`0x01`, `HeaderSize`, `Version`; (2) generate UUIDv4 in RFC 9562 binary layout; (3) compute BLAKE3 `PrevHash` (zero when encrypted); (4) set reserved fields to zero; (5) pad8 all variable fields; (6) never modify written blocks; (7) set `CapBitmap` accurately; (8) if importing: set `Flags` bit 0, emit Import Provenance as Block 0; (9) set `PayloadVersion` to 1; (10) set `CompAlgo` bits to 00 when uncompressed; (11) split regions exceeding $2^{32}-81$ bytes; (12) write 32 zero bytes for unavailable `DiskHash`; (13) reject `PageSizeLog2` outside $[10, 40]$; (14) emit well-formed UTF-8 per RFC 3629 for all string fields; (15) convert port numbers from network byte order to little-endian before writing; (16) set `BlockCount` to the total number of blocks or zero if unknown.

A conformant acquirer **MUST** emit Process Identity as Block 0 and Module List Index as Block 1.

Investigation: (17) emit System Context as Block 2; (18) set `CapBitmap` bit 11; (19) set `TableBitmap`; (20) set `Redacted` flag (bit 3) when any field has been intentionally stripped or pseudonymized.

Encryption: (21) `HeaderSize`=128; (22) valid encryption extension; (23) AEAD with header (+KEM ct) as AAD; (24) append 16-byte tag; (25) zero all `PrevHash`; (26) `Nonce` from CSPRNG; (27) KEM ciphertext after header when `KeyEncap`≠`0x00`.

Compression: (28) when `Compressed` is set, write 8-byte `UncompressedSize` before compressed data; (29) set `BlockLength` to on-disk size.

Block types: (30) **MUST NOT** emit block types whose payload format is not yet specified in this document. **Hash algorithm:** (31) set `HashAlgo` to a registered code from Table 12; the default (`0x00`, BLAKE3) **SHOULD** be used unless FIPS compliance, organizational policy, or platform performance measurements indicate otherwise; (32) use the selected algorithm exclusively for all `PrevHash`, `FileHash`, and `DiskHash` computations within the file; **MUST NOT** mix algorithms within a single file.

A producer **SHOULD** emit an EoC block. `FileHash` **SHOULD** be computed over plaintext even when encrypted. When encrypted, the producer **SHOULD** record an external file hash for chain-of-custody.

14.2 Consumer

A conformant consumer **MUST**: (1) reject invalid magic or `Endianness`≠`0x01`; (2) use `HeaderSize` to locate blocks; (3) skip unknown types via `BlockLength`; (4) reject `BlockLength` < 80; (5) detect end-of-blocks; (6) ignore reserved fields; (7) reject type `0x0000`; (8) skip unrecognized `PayloadVersion` for known block types with a logged warning indicating which block types were affected; (9) treat all-zero `DiskHash` as unavailable; (10) reject `PageSizeLog2` outside $[10, 40]$; (11) use explicit length fields to determine string extent, not NUL-scanning beyond the declared length; (12) preserve raw bytes for malformed UTF-8 fields and not reject the entire block; (13) if `BlockCount`>0, verify parsed count matches; (14) treat bytes after a verified EoC as trailing non-MSL data.

Encryption: (15) verify `HeaderSize`=128; read KEM ct; decrypt via AEAD with AAD; reject on failure; (16) skip `PrevHash` verification.

Investigation: (17) expect Block 2 = System Context; check `TableBitmap`; (18) when `Redacted` is set, interpret zero-length optional strings as intentionally redacted rather than absent.

Compression: (19) read 8-byte `UncompressedSize` before inflating; (20) reject if decompressed size does not match `UncompressedSize`.

Cross-references: (21) log and continue on dangling `ParentUUID` or payload-embedded UUID references; treat the affected block as an orphan but **MUST NOT** discard it. **Hash algorithm:** (22) **MUST** implement at minimum `HashAlgo=0x00` (BLAKE3) and `HashAlgo=0x01` (SHA-256); **MAY** implement `0x02` (SHA-512/256) and other registered codes; **MUST** reject files whose `HashAlgo` the consumer does not implement, emitting a diagnostic indicating the unsupported algorithm code.
A consumer **SHOULD**: verify BLAKE3 chain when unencrypted; verify EoC `FileHash` after decryption.

References

[1] S. Bradner, “Key words for use in RFCs to Indicate Requirement Levels,” RFC 2119, 1997.

[2] K. Davis, B. Peabody, P. Leach, “Universally Unique IDentifiers (UUIDs),” RFC 9562, 2024.

[3] F. Yergeau, “UTF-8, a transformation format of ISO 10646,” RFC 3629, 2003.

[4] J. O’Connor et al., “BLAKE3: One function, fast everywhere,” 2020. https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf

[5] NIST, “Secure Hash Standard (SHS),” FIPS 180-4, 2015. https://doi.org/10.6028/NIST.FIPS.180-4

[6] TIS Committee, “ELF Specification,” v1.2, 1995.

[7] NIST, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” FIPS 203, 2024. https://doi.org/10.6028/NIST.FIPS.203

[8] N. Bindel, J. Brendel, M. Fischlin, B. Goncalves, D. Stebila, “Hybrid Key Encapsulation Mechanisms and Authenticated Key Exchange,” in *Post-Quantum Cryptography (PQCrypto)*, Springer, 2019.

[9] D. Stebila, S. Fluhrer, S. Gueron, “Hybrid key exchange in TLS 1.3,” Internet-Draft draft-ietf-tls-hybrid-design, IETF, 2024.